MIVA OPEN UNIVERSITY

Department of Computing

IFT 211

Digital Logic Design — Mid-Semester Lab Assessment

Smart Water Tank Monitoring & Control System

Design, Boolean derivation and simulation in Logisim-evolution (pure digital logic)

Student	**Martins Sopuluchukwu Ngene**
Matric Number	**2025/B/CSC/0429**
Email	**m.ngene6895@miva.edu.ng**
Programme	**B.Sc. Computer Science**
Course	**IFT 211**
Submission type	**Individual lab assessment**
Simulator	**Logisim-evolution 3.9.0**
Circuit file	**SmartWaterTank_Martins_Ngene.circ**
GitHub	**github.com/martins-ngene/ift-211-smart-water-tank-system**
Date	**November 2026**

Contents

1. Project overview and design assumptions
2. System block diagram and top-level circuit
3. Step 1 — Sensor encoder (truth table, K-maps, Boolean derivation)
4. Step 2 — Full comparator
5. Step 3 — Pump control logic
6. Step 4 — Level register (D flip-flops)
7. Step 5 — Overflow counter and alarm
8. Step 6 — 2-bit-to-BCD mapping and BCD-to-7-segment decoder
9. LOW / MEDIUM / FULL status indicator
10. Testing scenarios and simulation results
11. How to run the simulation
12. Conclusion

Appendix — Full-size circuit screenshots (Figures A1–A13)

Deliverables checklist (Section 7 of the brief)

Required item	Where it is
Logisim/CircuitVerse circuit file	SmartWaterTank_Martins_Ngene.circ (Logisim-evolution 3.9.0), submitted with this report
Boolean derivations	Sections 3–9
Truth tables	Sections 3–9 (and simulator-generated tables in §10)
K-map simplifications	Sections 3, 7 and 8
Testing scenarios	Section 10 (17-step sequential test + exhaustive combinational tables)
System explanation	Sections 1, 2 and 12
Circuit screenshots	Appendix: full-page landscape screenshots of every circuit and 4 simulation states (Figures A1–A13)

1. Project overview and design assumptions

The system monitors a residential or industrial water tank using four level sensors, shows the tank level (0–3) on a 7-segment display, switches the filling pump ON while the tank is below FULL and OFF once it is FULL, lights LOW / MEDIUM / FULL indicators, and raises an overflow ALARM if the FULL signal stays high for three consecutive clock cycles. The whole design is built in Logisim-evolution from basic gates (NOT, AND, OR, NAND, XNOR), D flip-flops and a 7-segment display only — no microcontroller, no ROM/look-up tables, and none of Logisim's built-in encoder, decoder, comparator, counter or register blocks. Each stage is a sub-circuit drawn at gate level so the top-level schematic matches the block diagram in the brief.

Inputs and outputs

Signal	Type	Meaning
S0	Input pin	Bottom probe (tank effectively empty, below 25%)
S1	Input pin	25% probe
S2	Input pin	50% probe
S3	Input pin	100% (FULL) probe
CLK	Clock	System clock for the register and the overflow counter (rising-edge triggered)
RESET	Input pin	Asynchronous, active-high reset of all four flip-flops
7-segment display	Output	Stored tank level 0, 1, 2 or 3
PUMP_ON	LED	1 = pump running (tank below FULL)
FULL_DETECT	LED	Comparator output, LEVEL = 11
ALARM	LED	Overflow risk: FULL high for 3 clock cycles
LOW / MEDIUM / FULL	LEDs	Level status from the stored level
SENSOR_FAULT	LED	Impossible sensor pattern detected (see below)
CNT_Q1, CNT_Q0	LEDs	Overflow-counter state, shown for demonstration

Design assumptions

- A sensor outputs 1 when water touches its probe. Because water fills from the bottom, a healthy tank always gives a "thermometer" pattern: S3=1 implies S2=1, S2=1 implies S1=1, and S1=1 implies S0=1.
- The level is set by the highest wet probe (priority encoding): S3 → 11, S2 → 10, S1 → 01, otherwise 00. S0 alone (or no probe wet) is level 0.
- Any non-thermometer pattern (for example S2 wet but S1 dry) can only come from a stuck or failed sensor. The encoder flags it as SENSOR_FAULT, and the register refuses to load it, so the register really holds the "last valid tank level" (functional requirement 5).
- The pump and comparator act on the live encoded level so the pump stops at once when the tank fills. The display and status LEDs show the clocked, stored level.
- The overflow counter counts clock edges while FULL = 1, stops at 3 (so the alarm holds), and clears on the next clock edge after FULL drops to 0, as Step 5 of the brief requires.

2. System block diagram and top-level circuit

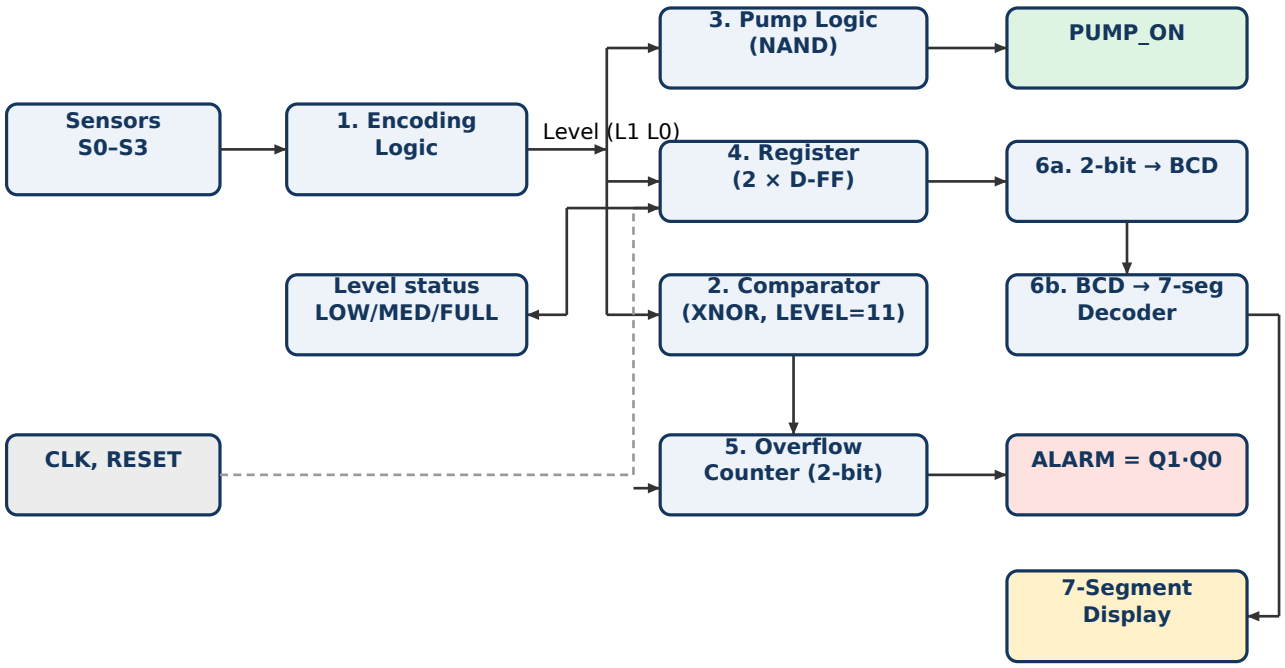


Figure 1 — System block diagram, following the transitions in Section 6 of the brief. Dashed lines are the clock and reset.

Figure 2 is the top-level Logisim circuit (**main**). Each numbered box is a gate-level sub-circuit; their schematics are in the Appendix. The tunnels sa…sg carry the decoder outputs to the 7-segment display.

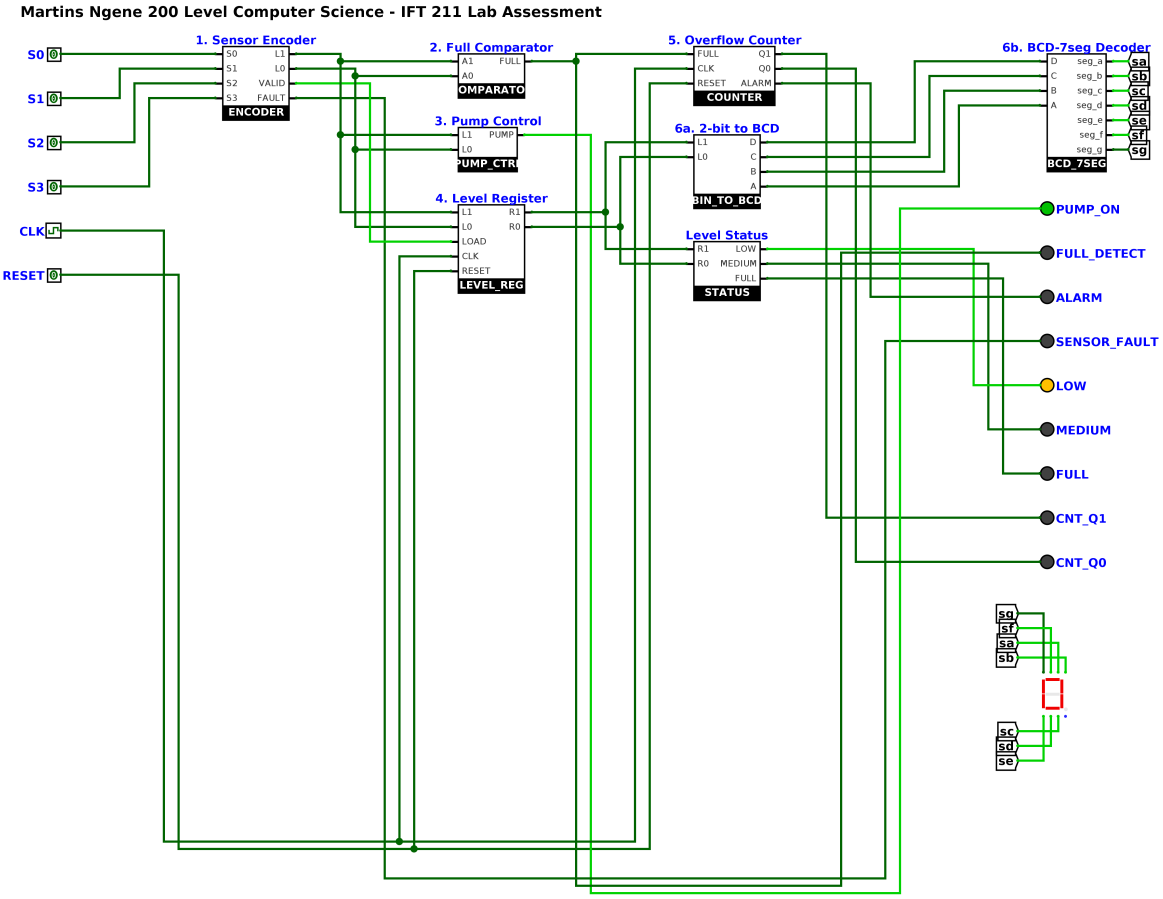


Figure 2 — Top-level circuit *main* in Logisim-evolution (print view).

3. Step 1 — Sensor encoder (combinational logic)

The encoder converts the four sensor bits into the 2-bit level code L1 L0 and also produces VALID/FAULT. Table 1 is the full 16-row truth table; the four rows with a valid (thermometer) pattern are shaded.

S3	S2	S1	S0	Pattern	L1	L0	Level	FAULT	VALID
0	0	0	0	valid	0	0	Empty	0	1
0	0	0	1	valid	0	0	Empty	0	1
0	0	1	0	fault	0	1	25%	1	0
0	0	1	1	valid	0	1	25%	0	1
0	1	0	0	fault	1	0	50%	1	0
0	1	0	1	fault	1	0	50%	1	0
0	1	1	0	fault	1	0	50%	1	0
0	1	1	1	valid	1	0	50%	0	1
1	0	0	0	fault	1	1	Full	1	0
1	0	0	1	fault	1	1	Full	1	0
1	0	1	0	fault	1	1	Full	1	0
1	0	1	1	fault	1	1	Full	1	0
1	1	0	0	fault	1	1	Full	1	0
1	1	0	1	fault	1	1	Full	1	0
1	1	1	0	fault	1	1	Full	1	0
1	1	1	1	valid	1	1	Full	0	1

Table 1 — Encoder truth table (matches the table Logisim generated in tty-table mode).

K-map simplification (rows S3 S2, columns S1 S0, Gray-code order)

K-map: L1

S3S2↓ S1S0→	00	01	11	10
00	0	0	0	0
01	1	1	1	1
11	1	1	1	1
10	1	1	1	1

Groups: row S2=1 (8 cells) and rows S3=1 (8 cells).
L1 = S3 + S2

K-map: L0

S3S2↓ S1S0→	00	01	11	10
00	0	0	1	1
01	0	0	0	0
11	1	1	1	1
10	1	1	1	1

Groups: S3=1 rows (octet) and the quad S2'·S1 (rows 00 and 10, cols 11 and 10; the row-10 cells are already 1 from S3).
L0 = S3 + S2'·S1

K-map: FAULT

S3S2↓ S1S0→	00	01	11	10
00	0	0	0	1
01	1	1	0	1
11	1	1	0	1
10	1	1	1	1

Three quads: S3·S2' (row 10), S2·S1' (cols 00 and 01 of rows 01 and 11), S1·S0' (col 10).
FAULT = S3·S2' + S2·S1' + S1·S0', **VALID = (FAULT)'**

Why not use don't-cares? If the fault rows were treated as don't-cares, L1 would shrink to just S2. That saves one gate, but a stuck-low S2 would then hide a full tank and the pump would overflow it. I kept the full priority form (S3 overrides everything) because it fails safe, and I report the inconsistency separately on FAULT.

Boolean derivation

From the truth table, L1 is 1 whenever the 50% or 100% probe is wet: L1 = Σm(4–7, 8–15) = S3 + S2 (Table 1 rows with S3=1 or S2=1). L0 is 1 for level 01 (highest wet probe S1) or level 11 (S3): L0 = S3 + S3'·S2'·S1. By absorption (X + X'Y = X + Y), L0 = S3 + S2'·S1. The fault condition is the OR of the three “a higher probe is wet but the one below is dry” terms.

L1 = S3 + S2

L0 = S3 + S2'·S1

FAULT = S3·S2' + S2·S1' + S1·S0' ; VALID = FAULT'

Gate count: 3 NOT, 4 AND, 3 OR (one 3-input), 1 NOT for VALID. The full-size encoder screenshot is Figure A2 in the Appendix.

4. Step 2 — Full comparator (LEVEL = 11)

A 2-bit equality comparator compares A = L1 L0 with the constant B = 11 (two Logisim *Constant* sources tied to 1). Each bit pair goes through an XNOR gate (output 1 when the bits are equal), and the two results are ANDed:

FULL = (A1 ⊙ B1)·(A0 ⊙ B0) = (L1 ⊙ 1)·(L0 ⊙ 1) = L1·L0

because x ⊙ 1 = x. So the general comparator reduces to the expression FULL = L1 · L0 given in the brief, while still using real comparator hardware.

L1 (A1)	L0 (A0)	A1⊙1	A0⊙1	FULL
0	0	0	0	0
0	1	0	1	0
1	0	1	0	0
1	1	1	1	1

Table 2 — Comparator truth table.

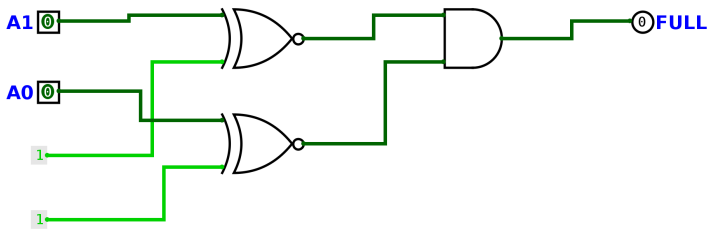


Figure 3 — FULL_COMPARATOR sub-circuit (XNOR equality comparator).

5. Step 3 — Pump control logic

The pump must run whenever the tank is below FULL and stop when it is FULL:

PUMP = FULL' = (L1·L0)' — one 2-input NAND gate

By De Morgan this is also L1' + L0', so the pump runs if either level bit is 0. A single NAND realises it directly.

L1	L0	Level	L1·L0	PUMP
0	0	0 (Empty)	0	1
0	1	1 (25%)	0	1
1	0	2 (50%)	0	1
1	1	3 (Full)	1	0

Table 3 — Pump control truth table.

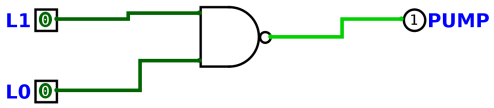


Figure 4 — PUMP_CTRL sub-circuit.

6. Step 4 — Level register (2 D flip-flops)

Two positive-edge-triggered D flip-flops (FF1, FF0) form a 2-bit parallel register that stores the level and synchronises it with CLK. To store the *last valid* level, each flip-flop input has a gate-level 2:1 multiplexer controlled by LOAD (= VALID from the encoder). When LOAD = 1 the new level is loaded; when LOAD = 0 the flip-flop is fed its own output and keeps its value.

D1 = LOAD·L1 + LOAD'·Q1 D0 = LOAD·L0 + LOAD'·Q0

Characteristic equation of a D flip-flop: Q(t+1) = D. So on each rising clock edge: Q(t+1) = L when LOAD = 1, and Q(t+1) = Q(t) when LOAD = 0. RESET drives the asynchronous reset input of both flip-flops, forcing Q1 Q0 = 00 at once.

RESET	CLK	LOAD	L1 L0	Q1 Q0 (next)	Action
1	x	x	xx	00	Asynchronous clear
0	↑	1	ab	ab	Load new valid level
0	↑	0	xx	Q1 Q0	Hold last valid level (sensor fault)
0	0 / 1 / ↓	x	xx	Q1 Q0	No change between edges

Table 4 — Level register function table.

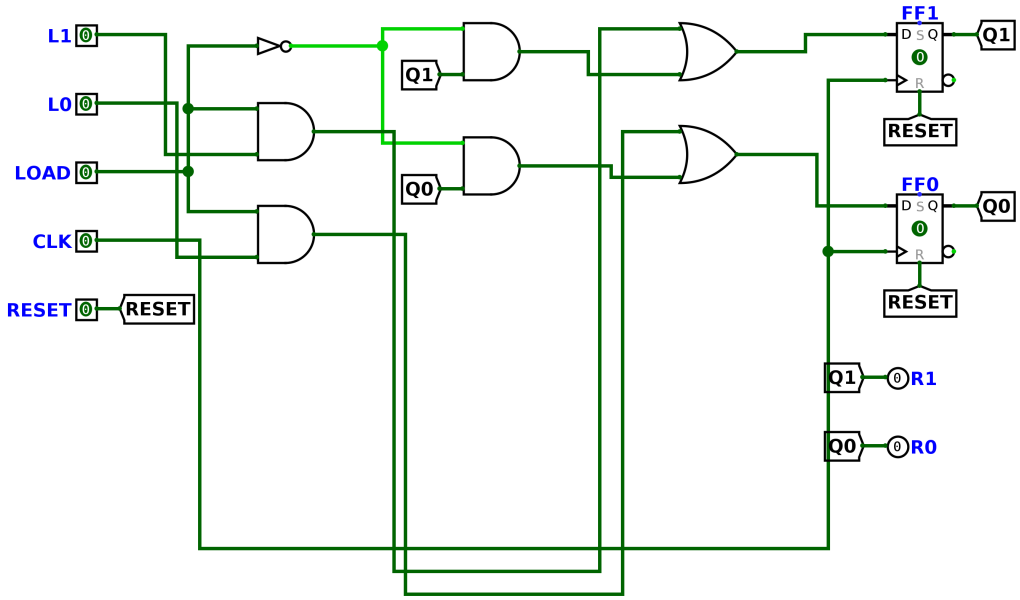


Figure 5 — LEVEL_REGISTER sub-circuit. The Q1/Q0 tunnels close the hold feedback loop.

This applies flip-flops, registers and clocking: both flip-flops share one clock, so the stored level changes only on rising edges and never glitches while the sensors settle.

7. Step 5 — Overflow counter and alarm (sequential logic)

A 2-bit synchronous counter (flip-flops C1, C0 sharing CLK) counts how many consecutive clock edges FULL has been high. It is enabled only when FULL = 1, clears synchronously when FULL = 0, and stops at 3 so the alarm stays on while the overflow risk continues. Table 5 is the state/excitation table. For D flip-flops the excitation input equals the next state.

FULL	Q1	Q0	$Q1^+$	$Q0^+$	D1	D0	Note
0	0	0	0	0	0	0	clear (FULL=0)
0	0	1	0	0	0	0	clear (FULL=0)
0	1	0	0	0	0	0	clear (FULL=0)
0	1	1	0	0	0	0	clear (FULL=0)
1	0	0	0	1	0	1	count 0→1
1	0	1	1	0	1	0	count 1→2
1	1	0	1	1	1	1	count 2→3
1	1	1	1	1	1	1	hold at 3 (ALARM)

Table 5 — Overflow counter state table.

K-map: D1

FULL↓ Q1Q0→	00	01	11	10
0	0	0	0	0
1	0	1	1	1

Pairs FULL·Q0 (cols 01, 11) and FULL·Q1 (cols 11, 10).
D1 = FULL·(Q1 + Q0)

K-map: D0

FULL↓ Q1Q0→	00	01	11	10
0	0	0	0	0
1	1	0	1	1

Pairs FULL·Q0' (cols 00, 10) and FULL·Q1 (cols 11, 10).
D0 = FULL·(Q1 + Q0')

D1 = FULL·Q1 + FULL·Q0 = FULL·(Q1 + Q0)

D0 = FULL·Q0' + FULL·Q1 = FULL·(Q1 + Q0')

ALARM = Q1·Q0

State sequence while FULL stays high: 00 → 01 → 10 → 11 → 11 … The alarm turns on at the third rising edge after the tank reaches FULL, so FULL has been high for three clock cycles (functional requirement 4). Any edge with FULL = 0 returns the counter to 00 (“counter resets when FULL = 0”), and RESET clears it asynchronously.

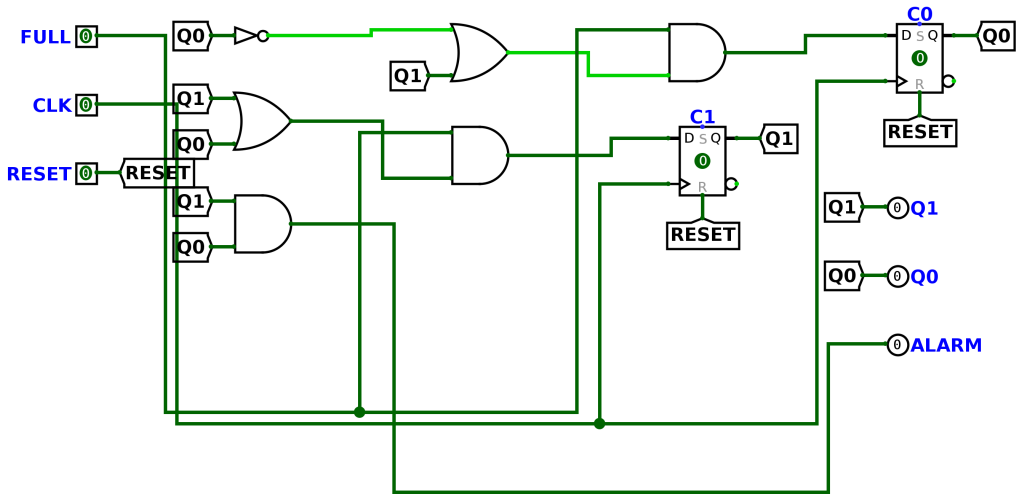


Figure 6 — OVERFLOW_COUNTER sub-circuit.

8. Step 6 — 2-bit-to-BCD mapping and BCD-to-7-segment decoder

2-bit-to-BCD mapping. The stored level R1 R0 (0–3) is a valid BCD digit once it is zero-extended: D = 0, C = 0, B = R1, A = R0. The BIN_TO_BCD sub-circuit does this with two constant-0 sources and direct wiring.

BCD-to-7-segment decoder. A full 4-input BCD decoder (common-cathode, active-high segments a–g) was designed with codes 10–15 as don't-cares (X), then built from gates. Table 6 is the decoder truth table.

Digit	D	C	B	A	a	b	c	d	e	f	g
0	0	0	0	0	1	1	1	1	1	1	0
1	0	0	0	1	0	1	1	0	0	0	0
2	0	0	1	0	1	1	0	1	1	0	1
3	0	0	1	1	1	1	1	1	0	0	1
4	0	1	0	0	0	1	1	0	0	1	1
5	0	1	0	1	1	0	1	1	0	1	1
6	0	1	1	0	1	0	1	1	1	1	1
7	0	1	1	1	1	1	1	0	0	0	0
8	1	0	0	0	1	1	1	1	1	1	1
9	1	0	0	1	1	1	1	1	0	1	1
–	1	0	1	0	X	X	X	X	X	X	X
–	1	0	1	1	X	X	X	X	X	X	X
–	1	1	0	0	X	X	X	X	X	X	X
–	1	1	0	1	X	X	X	X	X	X	X
–	1	1	1	0	X	X	X	X	X	X	X
–	1	1	1	1	X	X	X	X	X	X	X

Table 6 — BCD-to-7-segment truth table. Rows 0–3 (shaded) are the ones the tank system uses; 10–15 are don't-cares.

K-maps (rows D C, columns B A; X = don't-care)

K-map: segment a

DC↓ BA→	00	01	11	10
00	1	0	1	1
01	0	1	1	1
11	X	X	X	X
10	1	1	X	X

a = D (octet) + B (octet) + C·A (quad) + C'·A' (quad)

K-map: segment b

DC↓ BA→	00	01	11	10
00	1	1	1	1
01	1	0	1	0
11	X	X	X	X
10	1	1	X	X

b = C' (octet) + B'·A' (quad) + B·A (quad)

K-map: segment c

DC↓ BA→	00	01	11	10
00	1	1	1	0
01	1	1	1	1
11	X	X	X	X
10	1	1	X	X

c = C (octet) + B' (octet) + A (octet)

K-map: segment d

DC↓ BA→	00	01	11	10
00	1	0	1	1
01	0	1	0	1
11	X	X	X	X
10	1	1	X	X

d = D + C'·A' + C'·B + B·A' + C·B'·A

K-map: segment e

DC↓ BA→	00	01	11	10
00	1	0	0	1
01	0	0	0	1
11	X	X	X	X
10	1	0	X	X

e = C'·A' (quad) + B·A' (quad)

K-map: segment f

DC↓ BA→	00	01	11	10
00	1	0	0	0
01	1	1	0	1
11	X	X	X	X
10	1	1	X	X

f = D + B'·A' + C·B' + C·A'

K-map: segment g

DC↓ BA→	00	01	11	10
00	0	0	1	1
01	1	1	0	1
11	X	X	X	X
10	1	1	X	X

g = D + C·B' + C'·B + B·A'

Simplified segment equations and the tank-display case

Segment	General BCD form (implemented)	With D = C = 0 (levels 0–3)
a	D + B + C·A + C'·A'	B + A'
b	C' + B'·A' + B·A	1
c	C + B' + A	B' + A
d	D + C'·A' + C'·B + B·A' + C·B'·A	B + A'
e	C'·A' + B·A'	A'
f	D + B'·A' + C·B' + C·A'	B'·A'
g	D + C·B' + C'·B + B·A'	B

Table 7 — Decoder equations.

I implemented the general decoder rather than the reduced right-hand column, so the stage is a reusable BCD-to-7-segment decoder as the brief asks. The gate-level schematic is in the Appendix (Figure A8). Logisim's exhaustive truth table for the sub-circuit matches Table 6 for digits 0–9.

9. LOW / MEDIUM / FULL status indicator

Real-world requirement 2 asks the system to indicate LOW, MEDIUM or FULL. From the stored level R1 R0:

LOW = R1' (0% or 25%) MEDIUM = R1·R0' (50%) FULL = R1·R0 (100%)

R1	R0	Level	LOW	MEDIUM	FULL
0	0	0	1	0	0
0	1	1	1	0	0
1	0	2	0	1	0
1	1	3	0	0	1

Table 8 — Status indicator truth table.

10. Testing scenarios and simulation results

Testing was done in two layers, both on the submitted .circ file in Logisim-evolution 3.9.0:

- **Exhaustive combinational tests.** Logisim's command-line truth-table mode (*--toplevel-circuit <name> -tty table*) was run on every combinational sub-circuit (ENCODER, COMPARATOR, PUMP_CTRL, BIN_TO_BCD, BCD_7SEG, STATUS). All outputs matched Tables 1, 2, 3, 6 and 8 for every input combination.
- **Sequential system test.** The full *main* circuit was driven through the 17-step scenario below, toggling the real Clock component. After each step the LEDs and the seven segment inputs were read back from the simulator and compared with a separate Python model of the design equations.

#	Action / input	Clock	Expected: digit, pump, full, alarm, fault, count	Observed in Logisim	Result
0	RESET=1, all sensors dry	—	0, ON, 0, 0, 0, 0	0, ON, 0, 0, 0, 0	PASS
1	Release RESET, tank empty	1 edge	0, ON, 0, 0, 0, 0	0, ON, 0, 0, 0, 0	PASS
2	S0=S1=1 (25%)	1 edge	1, ON, 0, 0, 0, 0	1, ON, 0, 0, 0, 0	PASS
3	S2=1 (50%)	1 edge	2, ON, 0, 0, 0, 0	2, ON, 0, 0, 0, 0	PASS
4	S3=1 (FULL), before clock edge	none	2, OFF, 1, 0, 0, 0	2, OFF, 1, 0, 0, 0	PASS
5	FULL held	1st edge	3, OFF, 1, 0, 0, 1	3, OFF, 1, 0, 0, 1	PASS
6	FULL held	2nd edge	3, OFF, 1, 0, 0, 2	3, OFF, 1, 0, 0, 2	PASS
7	FULL held	3rd edge	3, OFF, 1, 1, 0, 3	3, OFF, 1, 1, 0, 3	PASS
8	FULL held (alarm latched at 3)	4th edge	3, OFF, 1, 1, 0, 3	3, OFF, 1, 1, 0, 3	PASS
9	S3=0 (drains to 50%), before edge	none	3, ON, 0, 1, 0, 3	3, ON, 0, 1, 0, 3	PASS
10	Drained to 50%	1 edge	2, ON, 0, 0, 0, 0	2, ON, 0, 0, 0, 0	PASS
11	S1=0 with S2=1: sensor fault, before edge	none	2, ON, 0, 0, 1, 0	2, ON, 0, 0, 1, 0	PASS
12	Fault held through a clock edge	1 edge	2, ON, 0, 0, 1, 0	2, ON, 0, 0, 1, 0	PASS
13	S1=1, S2=0: back to 25%	1 edge	1, ON, 0, 0, 0, 0	1, ON, 0, 0, 0, 0	PASS
14	RESET=1 (asynchronous)	none	0, ON, 0, 0, 0, 0	0, ON, 0, 0, 0, 0	PASS
15	RESET=0, S2=S3=1: full for only 2 cycles	2 edges	3, OFF, 1, 0, 0, 2	3, OFF, 1, 0, 0, 2	PASS
16	Drop and refill: counter restarts	1+2 edges	3, OFF, 1, 0, 0, 2	3, OFF, 1, 0, 0, 2	PASS

Table 9 — Sequential test scenario (17 checks, all PASS). The LOW/MEDIUM/FULL LEDs were also checked at every step. Digit = number shown on the 7-segment display.

Key behaviours confirmed

- **Pump cut-off is immediate:** in step 4, PUMP_ON goes OFF as soon as S3 = 1, before any clock edge, because the pump logic uses the live level.
- **Alarm after exactly 3 cycles:** the counter reads 1, 2, 3 over steps 5–7 and ALARM turns on at step 7. Step 15 shows that 2 cycles are not enough.

- **Counter reset when FULL = 0:** after draining (step 10) the counter returns to 0 and ALARM clears on the next edge. After a refill (step 16) it starts counting again from 0.
- **Last valid level kept:** during the sensor fault (steps 11–12) SENSOR_FAULT lights and the display stays at 2 instead of loading the bad reading.
- **Asynchronous reset:** step 14 clears the display and counter without a clock edge.

Simulation snapshots

Logisim colours wires bright green for 1 and dark green for 0. Figure 7 shows the system in the overflow state (step 7): the tank is FULL, the display shows 3, the pump is OFF, the counter is 11 and the ALARM LED is lit.

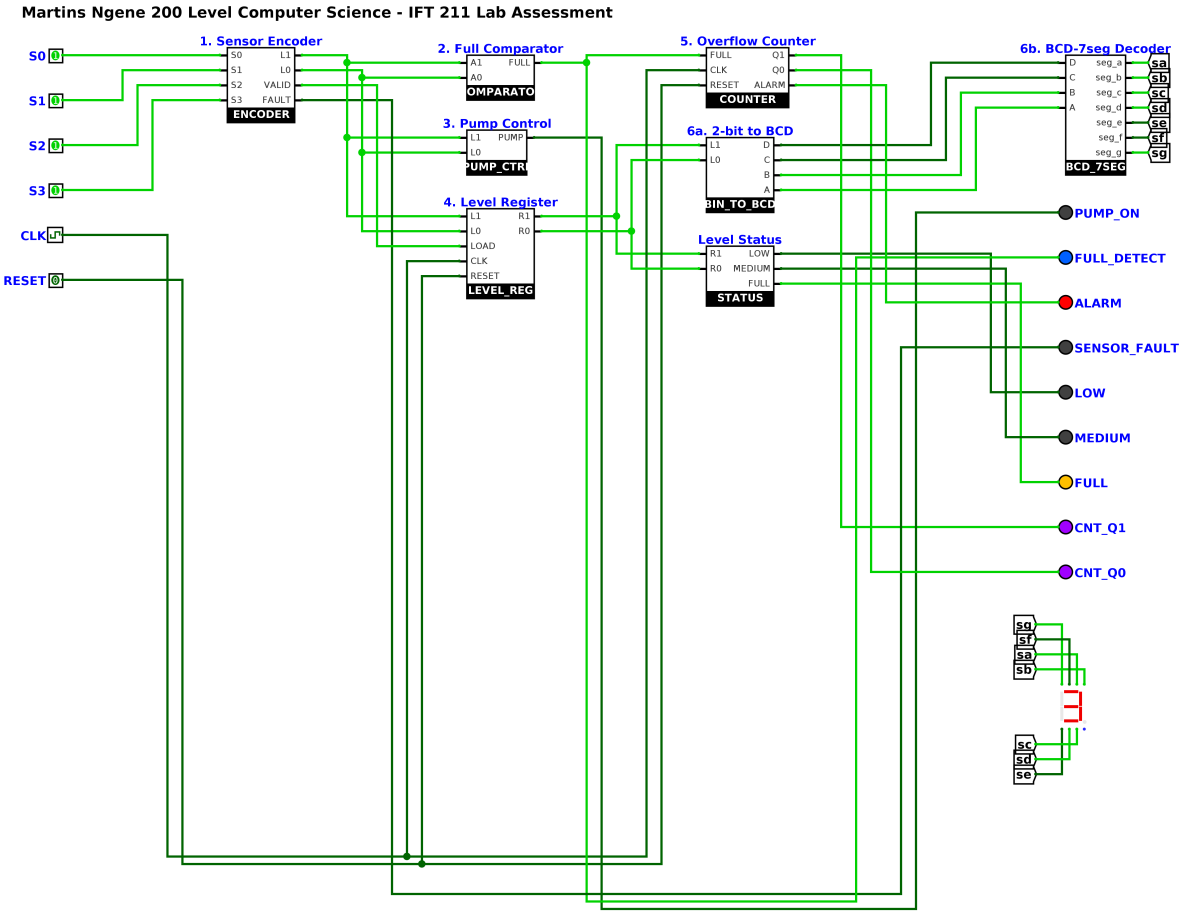


Figure 7 — Live simulation, FULL for three cycles → ALARM.

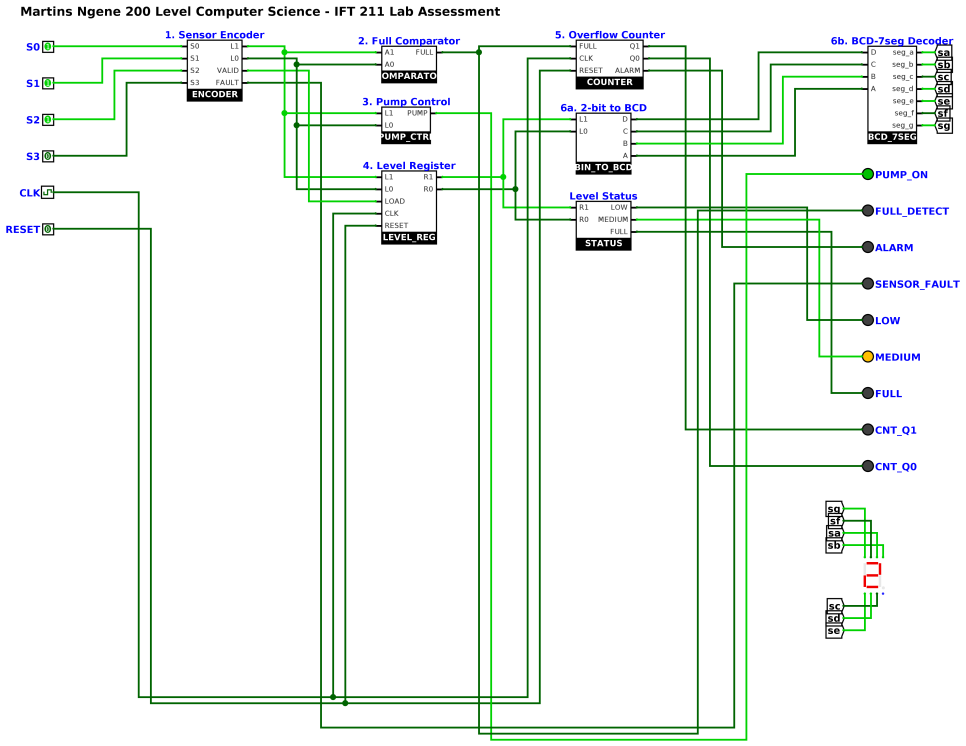


Figure 8 — Tank at 50%: display shows 2, MEDIUM on, pump ON.

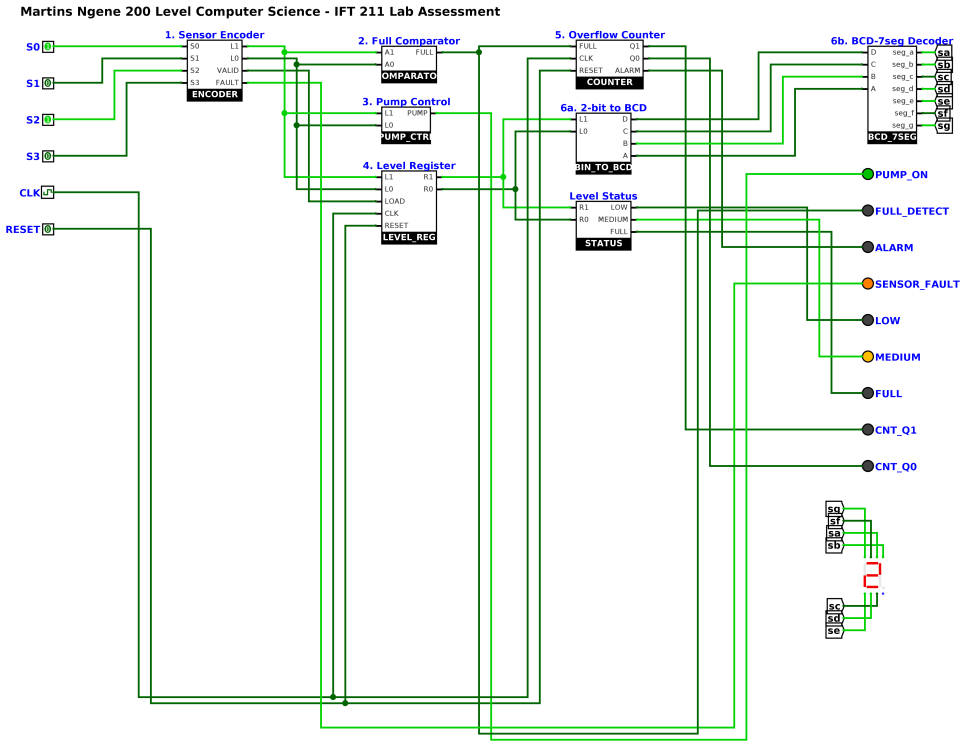


Figure 9 — Sensor fault (S2 wet, S1 dry): SENSOR_FAULT on, register holds the last valid level 2.

11. How to run the simulation

1. Open **SmartWaterTank_Martins_Ngene.circ** in Logisim-evolution 3.9.0 or newer (File → Open). The *main* circuit opens.
2. Select the Poke tool (hand icon). Click RESET to set it to 1, then back to 0, to clear the register and counter.
3. Enable the clock: Simulate → Auto-Tick Enabled (Ctrl+K), or press Ctrl+T to step one tick at a time.
4. Poke S0, S1, S2, S3 in order to fill the tank. Watch the 7-segment digit go 0 → 1 → 2 → 3 and the LOW/MEDIUM/FULL LEDs.
5. With S3 = 1, PUMP_ON switches off at once. After three rising clock edges ALARM lights (CNT_Q1/CNT_Q0 show the count).
6. Set S3 = 0 to simulate draining. On the next clock edge ALARM clears and the pump turns on.
7. Try an impossible pattern (e.g. S2 = 1, S1 = 0) to see SENSOR_FAULT light while the display keeps the last valid level.
8. Double-click any block (or pick it in the circuit list) to inspect its gate-level implementation.

The circuit file, this report and all screenshots are also available on GitHub: https://github.com/martins-ngene/ift-211-smart-water-tank-system

12. Conclusion

The Smart Water Tank Monitoring & Control System meets all five functional requirements using pure digital logic. Combinational design (priority encoder, XNOR comparator, NAND pump control, BCD decoder) was derived from truth tables and minimised with Karnaugh maps. Sequential design (a 2-bit D flip-flop register with load control and a 2-bit saturating synchronous counter) provides clock-synchronised level storage and the three-cycle overflow alarm. The design is organised to mirror the block diagram in the brief. It adds two safety features beyond the minimum: sensor-fault detection that protects the stored level, and a fail-safe priority encoder that never misses a FULL reading. Exhaustive truth-table checks and a 17-step clocked test in Logisim confirm that the circuit behaves as designed.

Concepts applied

Concept	Where used
Logic gates	Every block: NOT, AND, OR, NAND, XNOR
Boolean simplification	Absorption (encoder L0), De Morgan (pump), factoring (counter D-equations)
Encoders / decoders	Sensor priority encoder; BCD-to-7-segment decoder
7-segment display	Level display 0–3
Flip-flops	4 positive-edge D flip-flops with asynchronous reset
Registers	2-bit level register with load/hold mux
Counters	2-bit synchronous saturating overflow counter
Comparator	2-bit XNOR equality comparator (LEVEL = 11)

Appendix — Full-size circuit screenshots

These screenshots come from the submitted SmartWaterTank_Martins_Ngene.circ in Logisim-evolution 3.9.0. Each one gets its own landscape page so the gates, pin labels and wiring are readable. They were captured at 3× resolution, so you can zoom in. Wire colours show live logic values: bright green = 1, dark green = 0. Lit LEDs and segments are active. Schematics A2–A9 are shown with all inputs at 0 (constant sources show their fixed value).

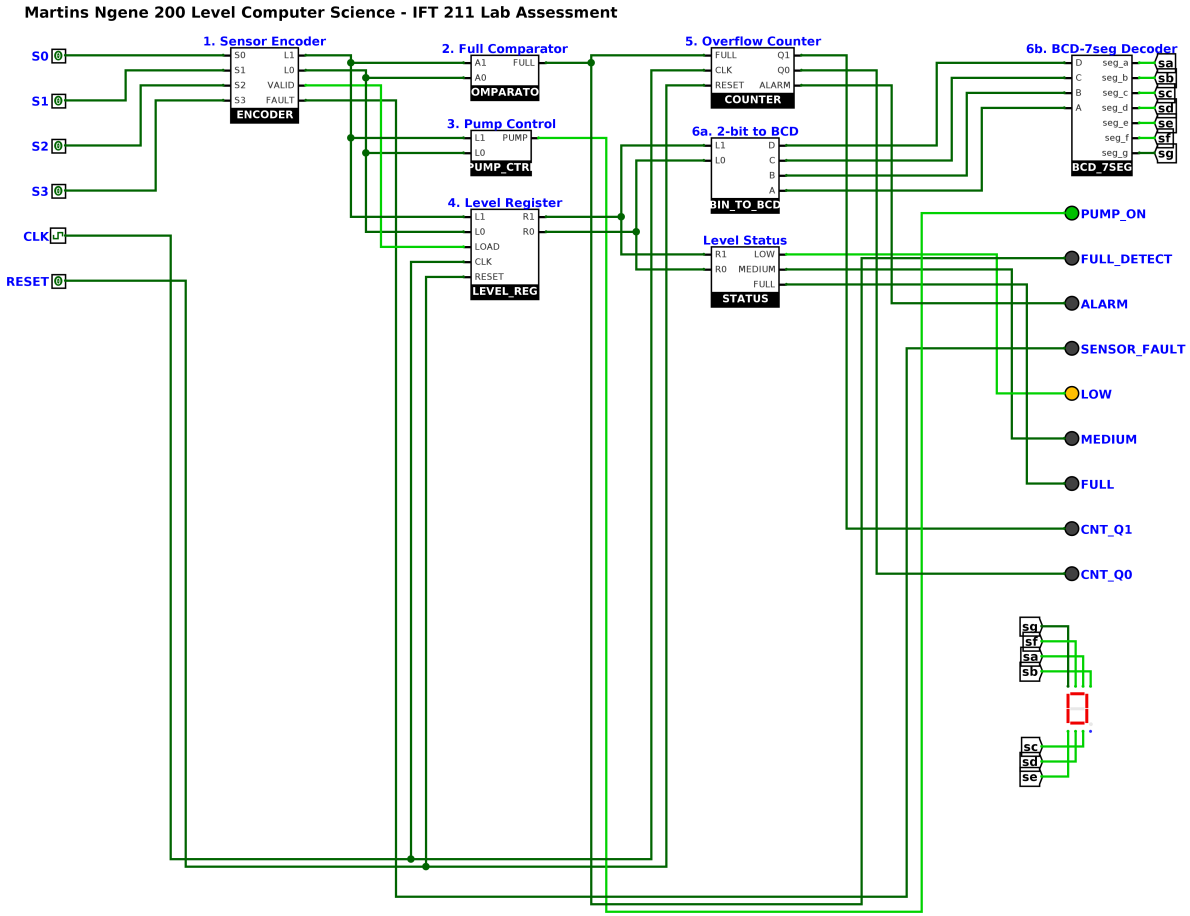


Figure A1 — Top-level circuit *main*: sensors, clock and reset feed the six numbered stages, LED indicators and 7-segment display.

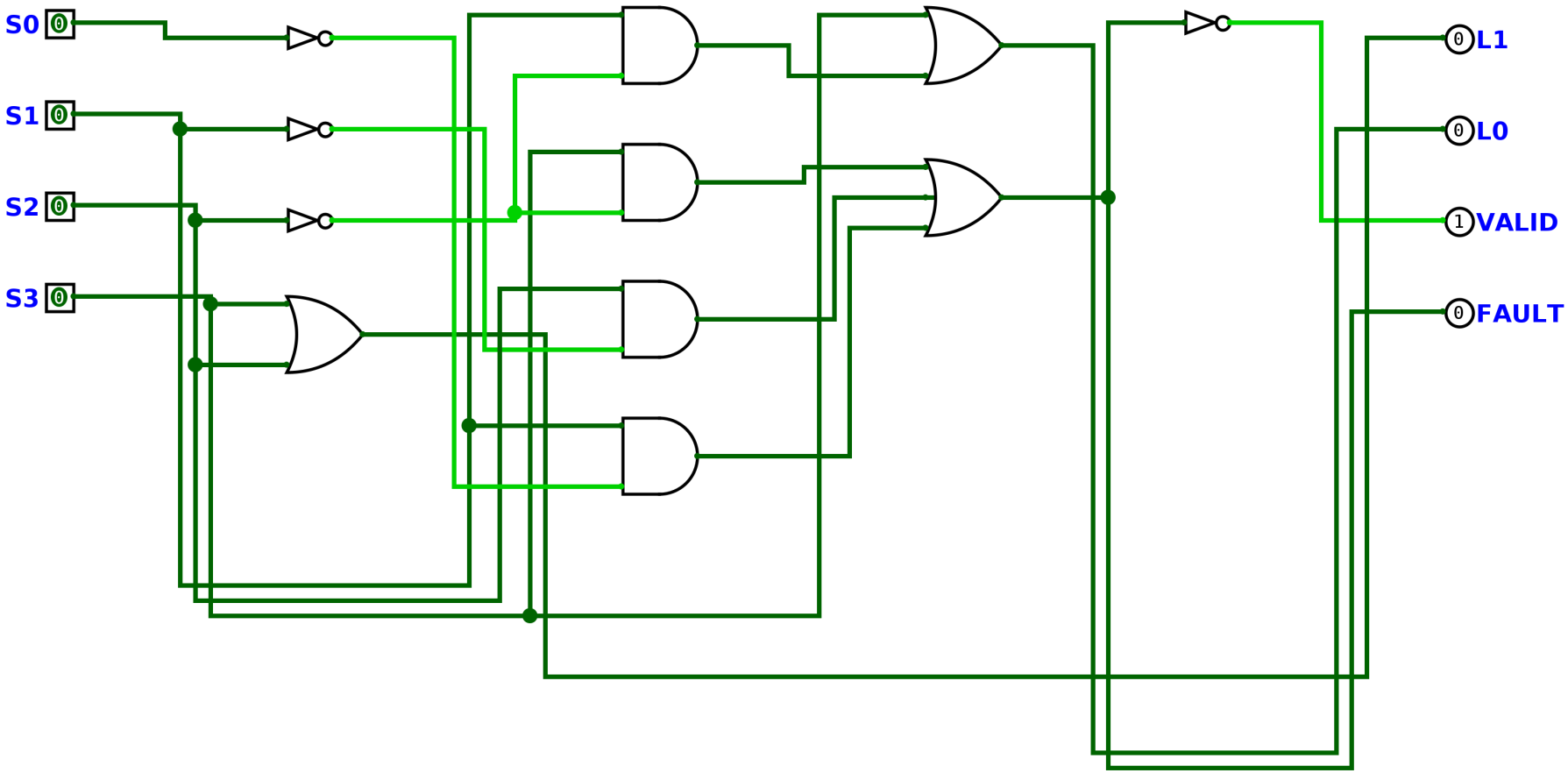


Figure A2 — ENCODER (Step 1): priority encoder L1 = S3 + S2, L0 = S3 + S2'·S1, plus FAULT/VALID detector.

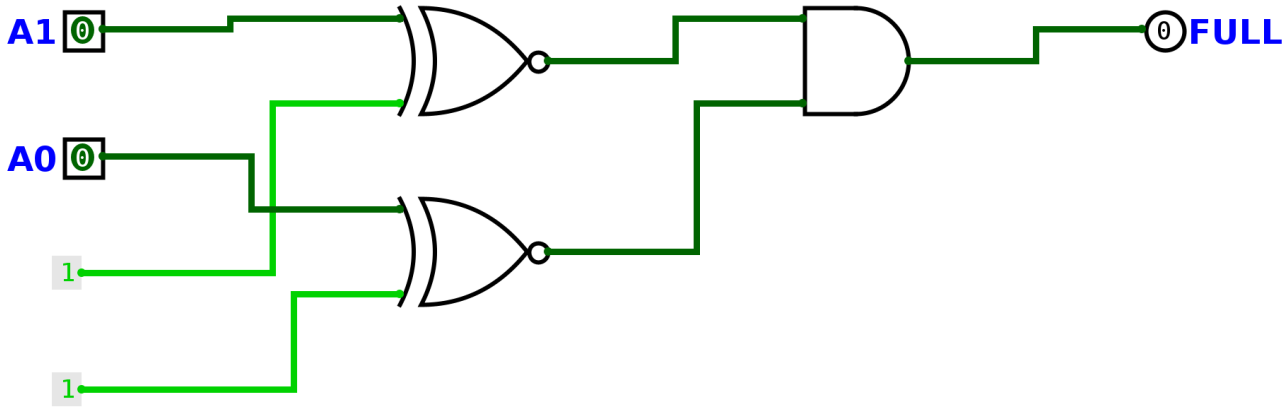


Figure A3 — COMPARATOR (Step 2): 2-bit XNOR equality comparator against constant 11, FULL = L1·L0.

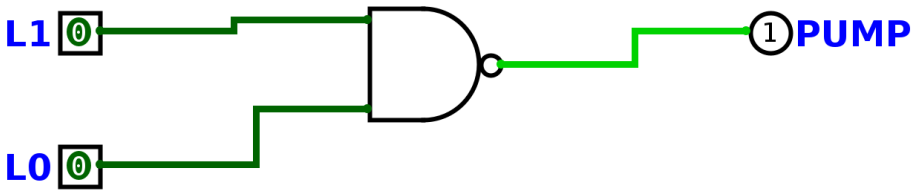


Figure A4 — PUMP_CTRL (Step 3): PUMP = (L1·L0)' using one NAND gate.

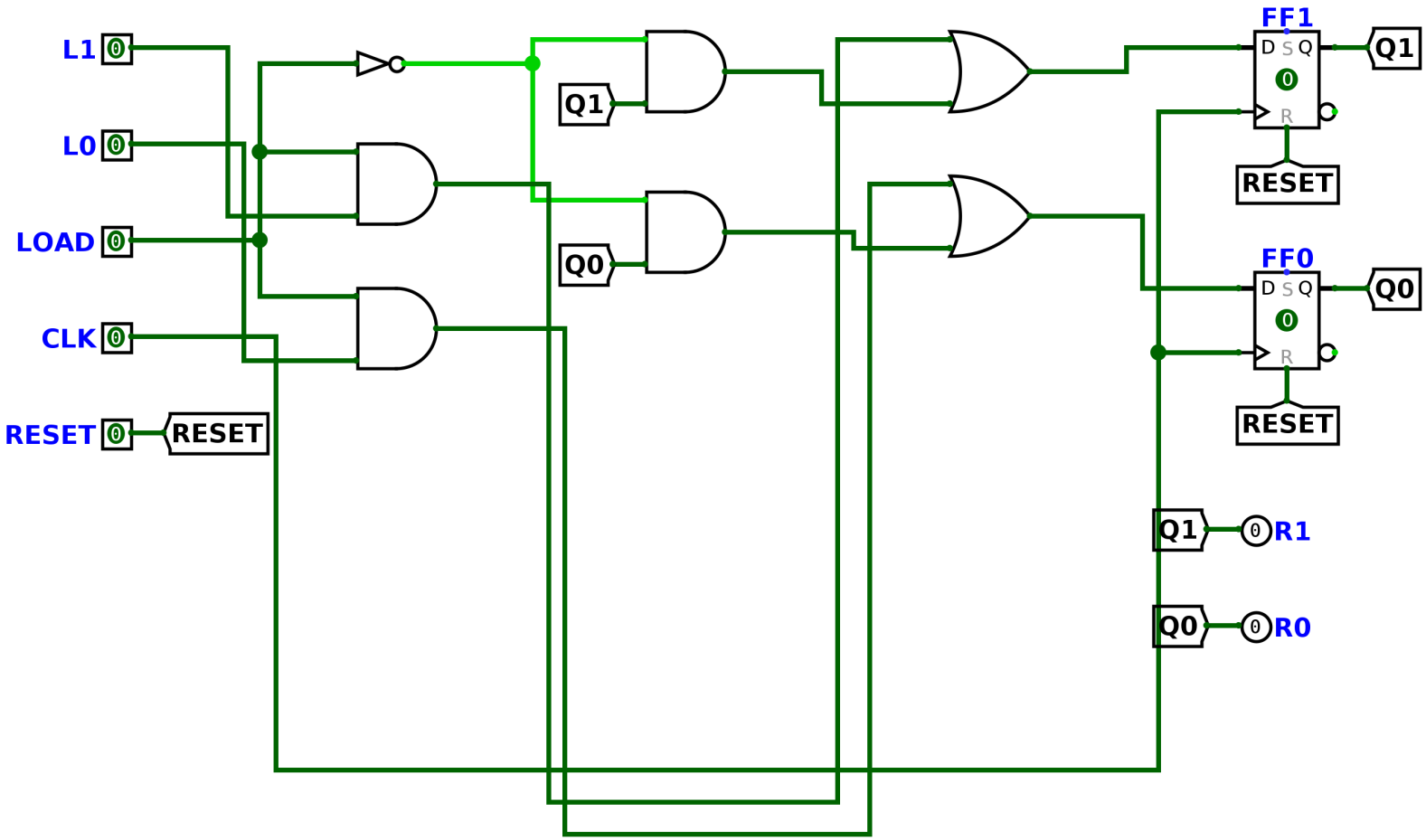


Figure A5 — LEVEL_REG (Step 4): two D flip-flops with load/hold multiplexers and asynchronous reset.

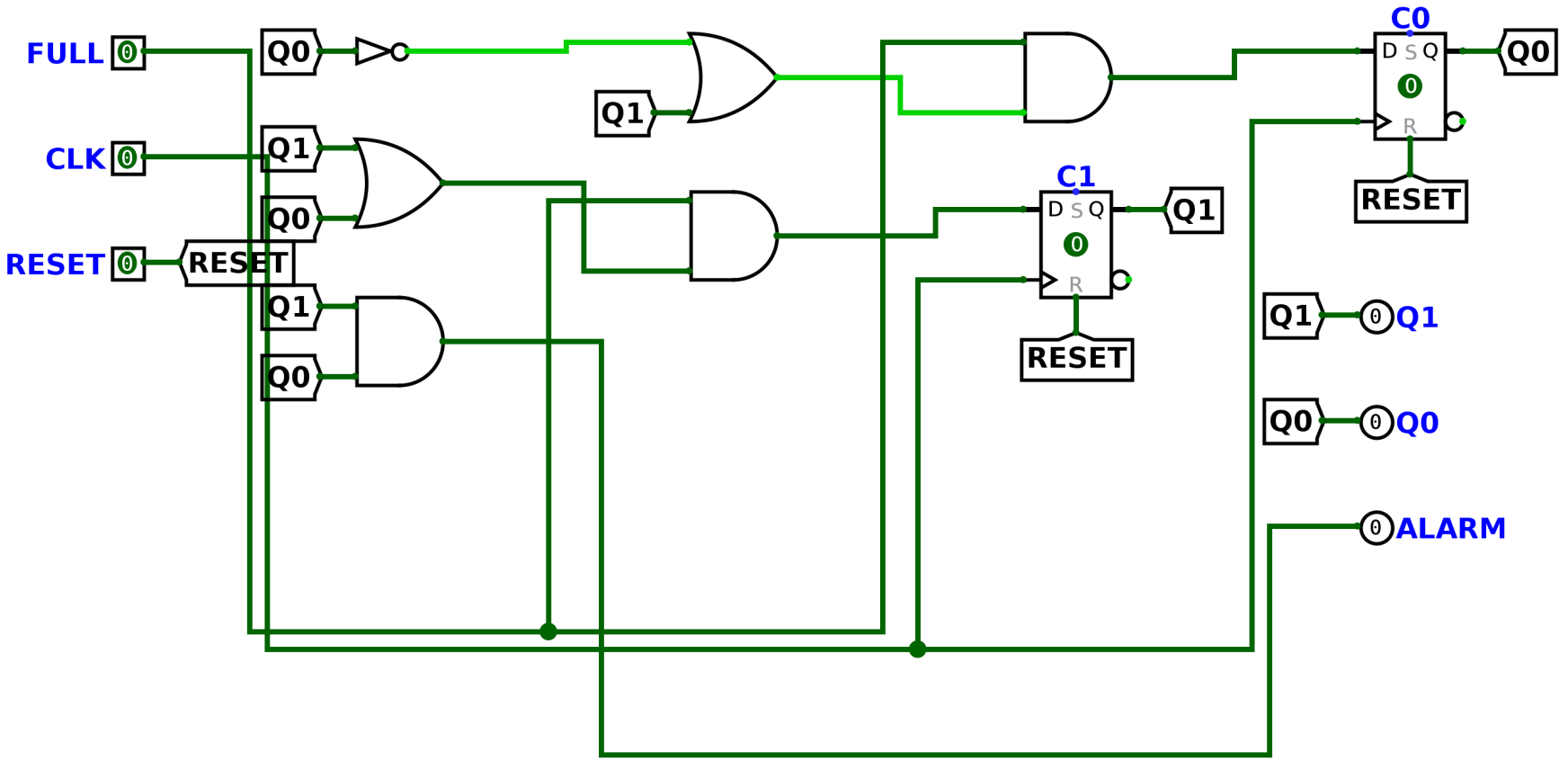


Figure A6 — COUNTER (Step 5): 2-bit synchronous saturating counter, D1 = FULL·(Q1+Q0), D0 = FULL·(Q1+Q0'), ALARM = Q1·Q0.

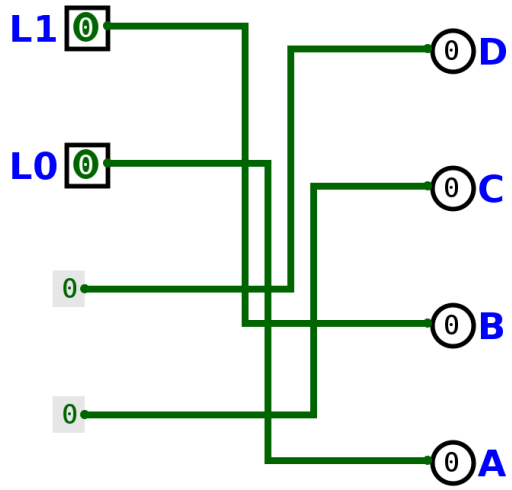


Figure A7 — BIN_TO_BCD (Step 6a): zero-extends the 2-bit level to BCD (D = C = 0, B = L1, A = L0).

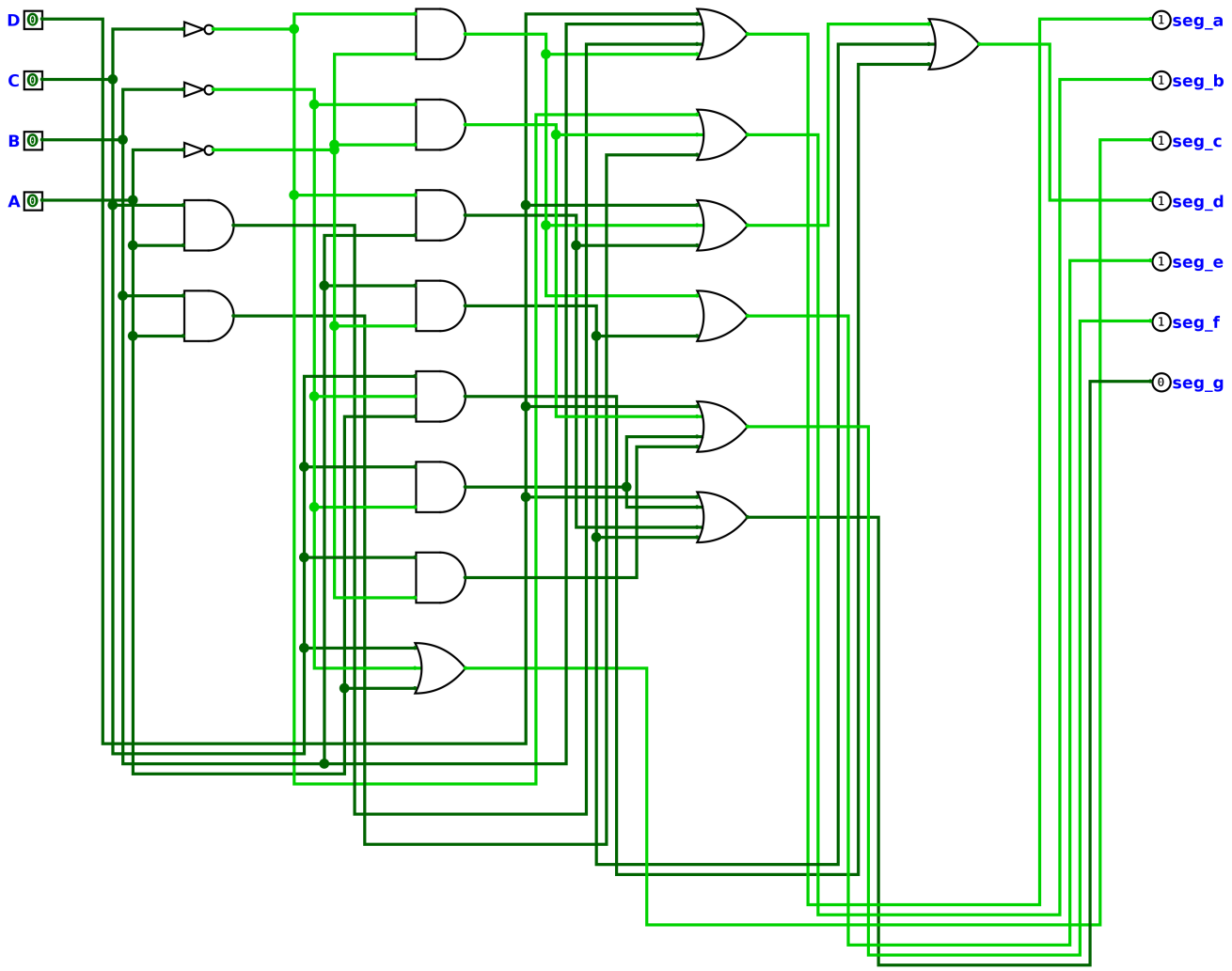


Figure A8 — BCD_7SEG (Step 6b): gate-level BCD-to-7-segment decoder (segment equations in Table 7).

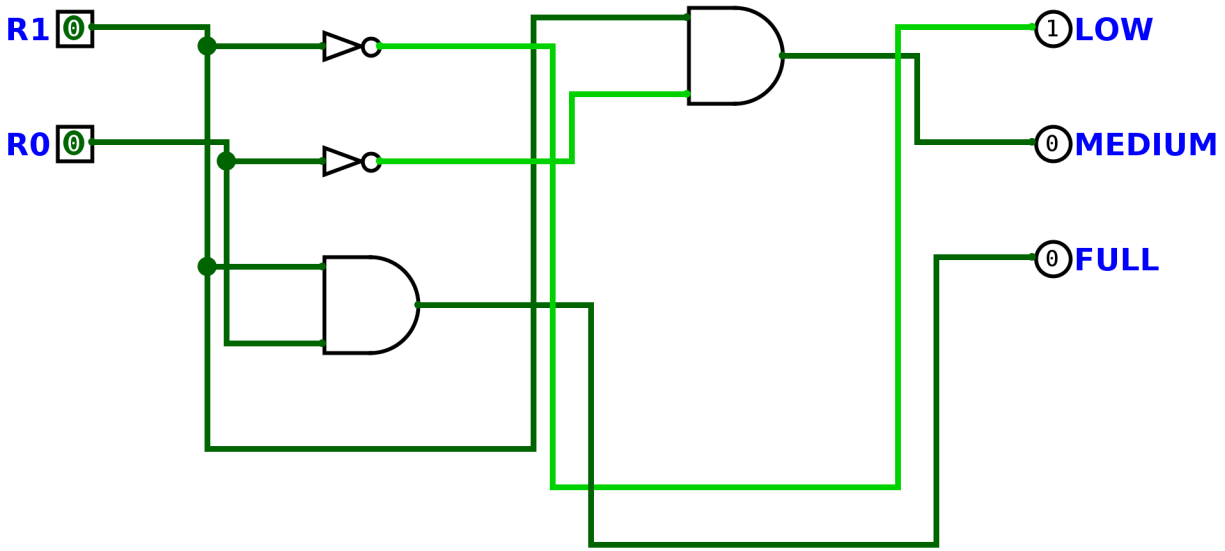


Figure A9 — STATUS: LOW = R1', MEDIUM = R1·R0', FULL = R1·R0.

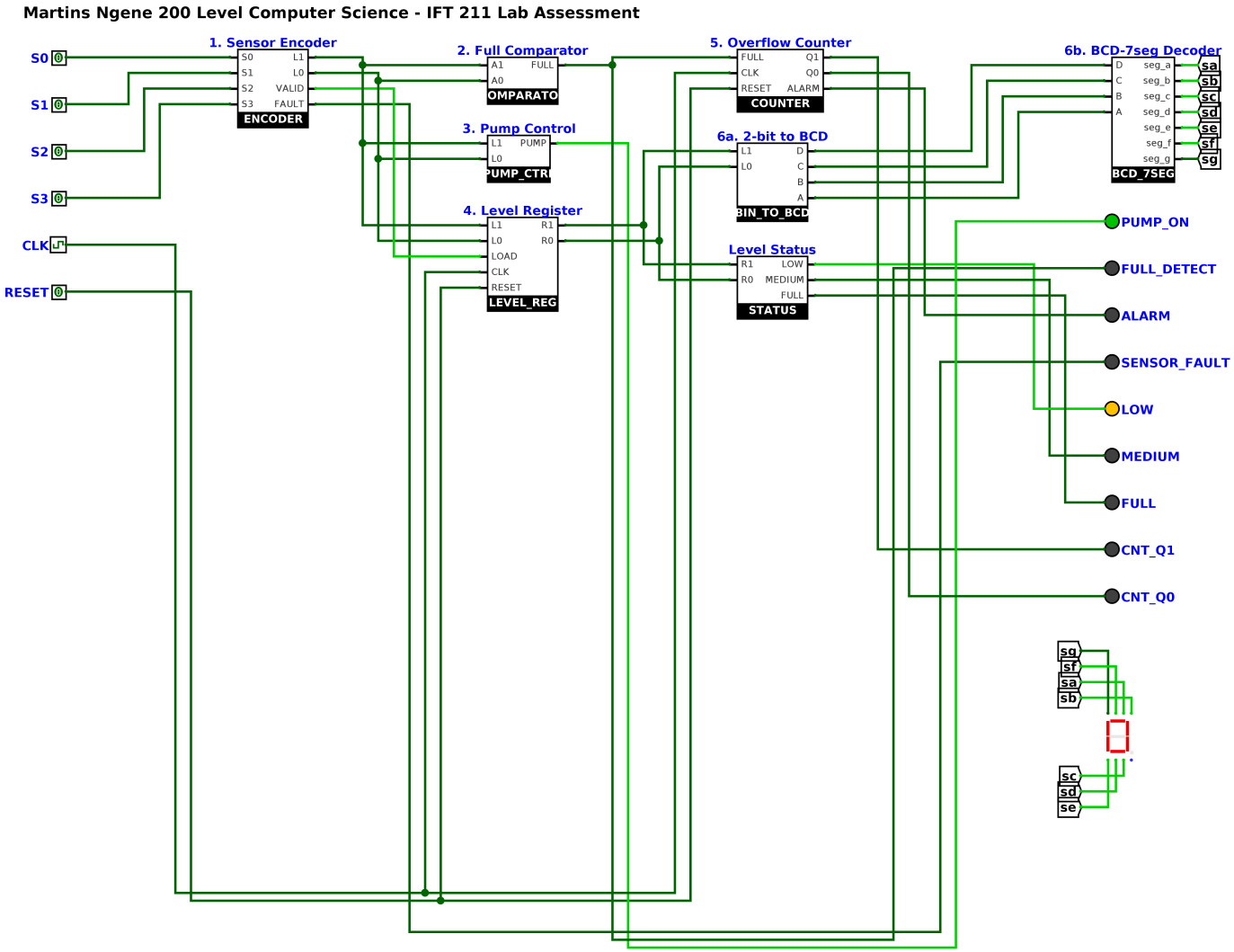


Figure A10 — Simulation: tank empty after reset. Display 0, LOW on, pump ON.

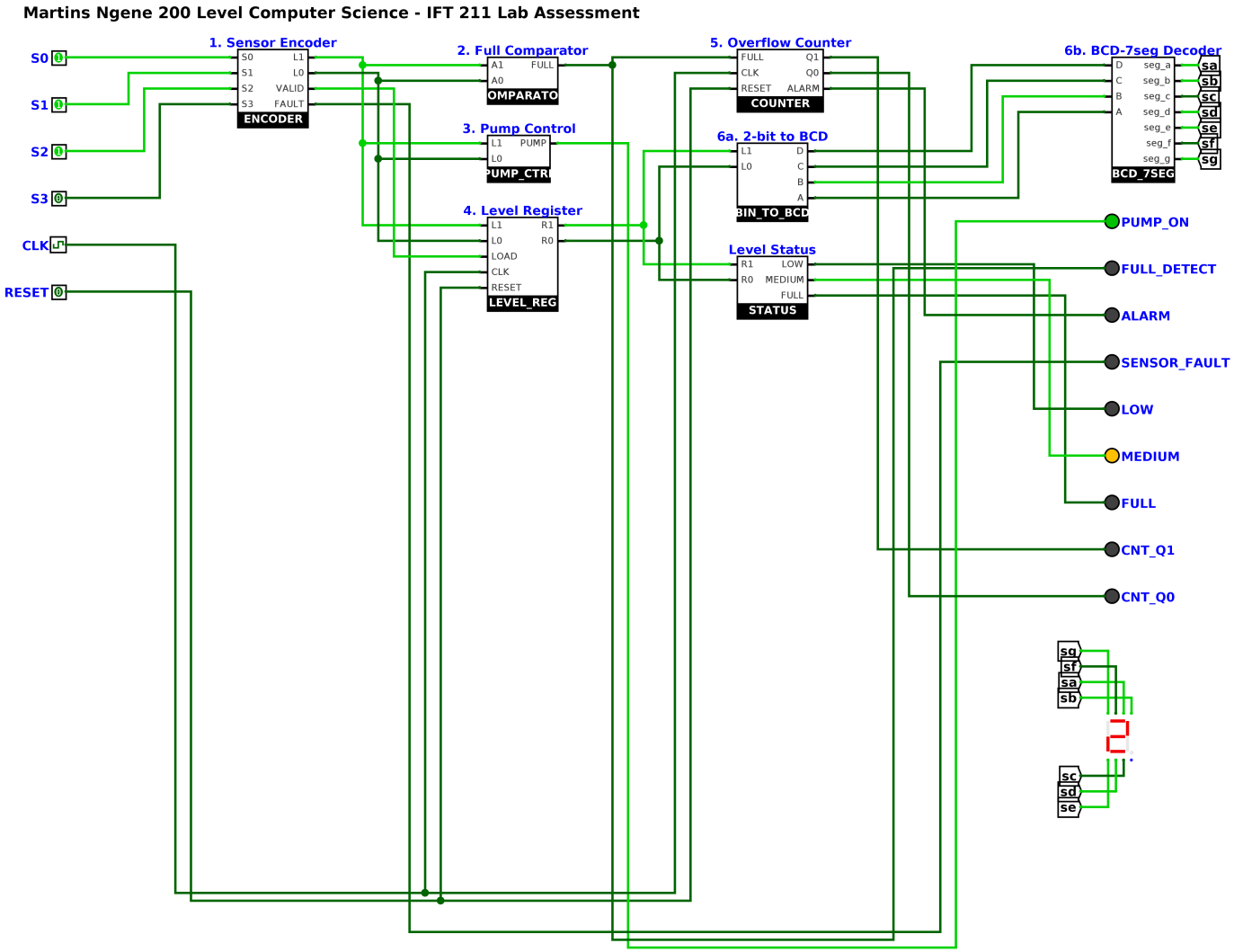


Figure A11 — Simulation: tank at 50%. Display 2, MEDIUM on, pump ON.

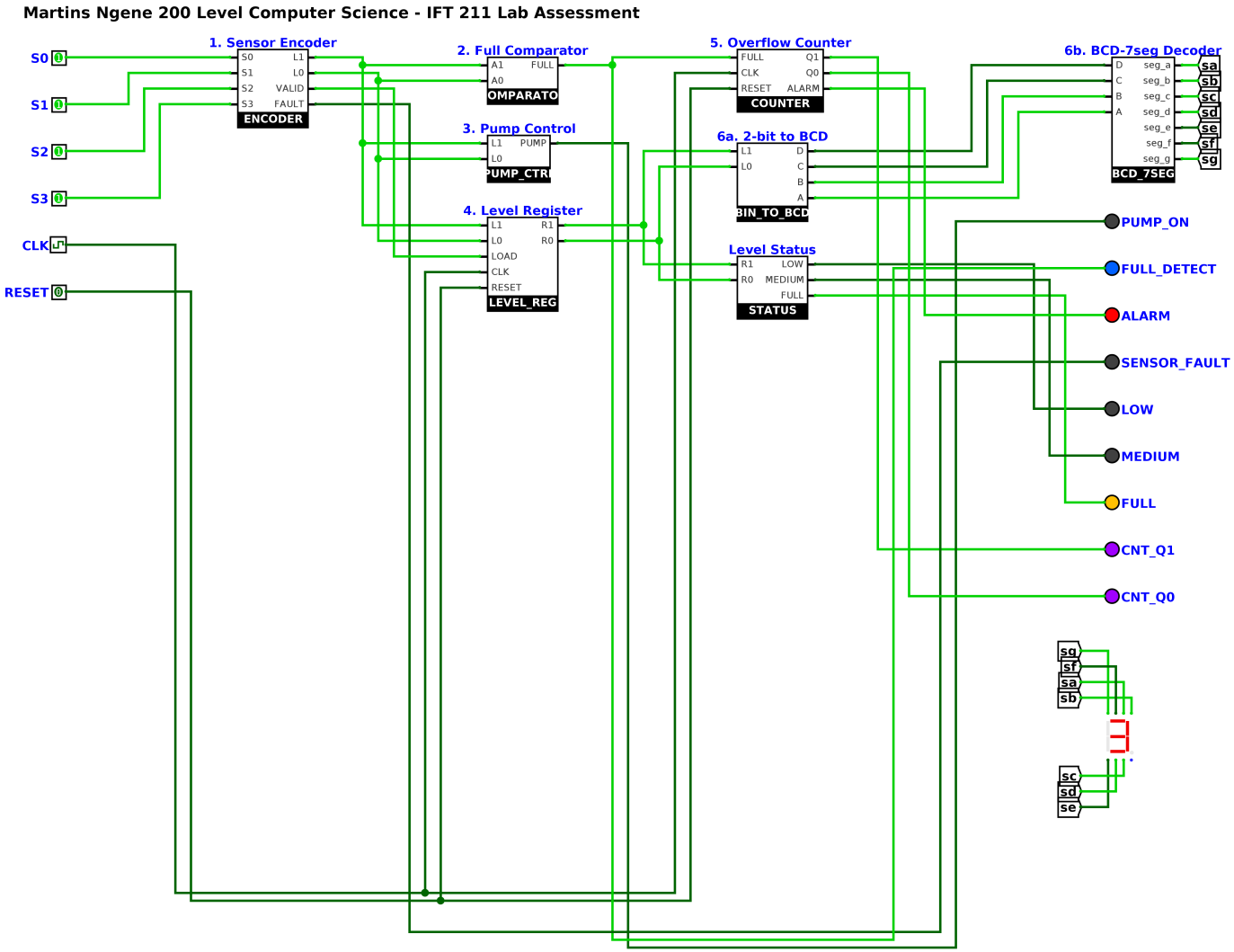


Figure A12 — Simulation: FULL for 3 clock cycles. Display 3, pump OFF, counter = 11, ALARM on.

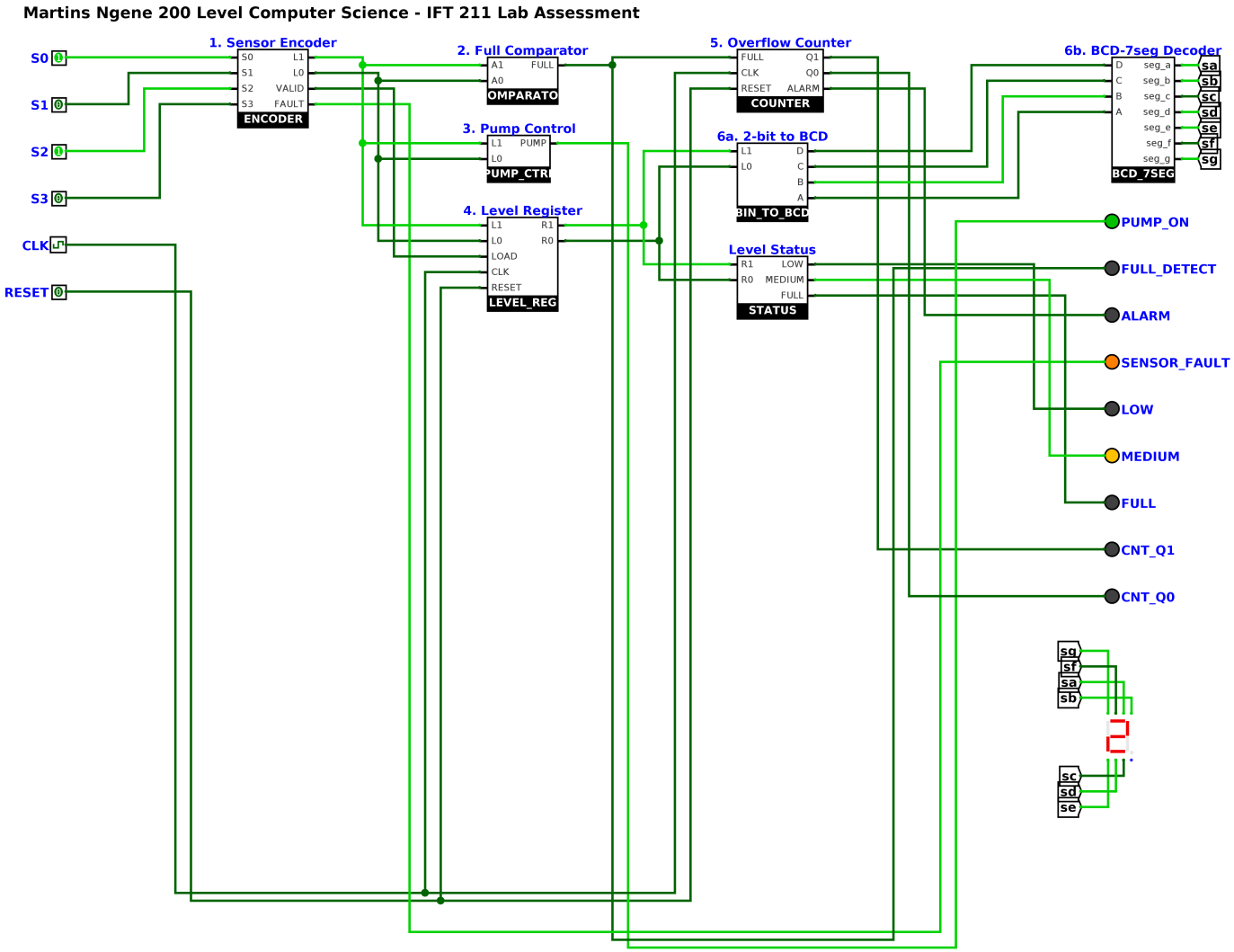


Figure A13 — Simulation: sensor fault (S2 wet, S1 dry). SENSOR_FAULT on, register keeps the last valid level 2.